

Priority scheduling non-pre-emptive

```
#include <stdio.h>

int main()
{
    int n;
    int pid[20], at[20], bt[20], pr[20];
    int ct[20], tat[20], wt[20];
    int completed[20] = {0};

    int current_time = 0, completed_count = 0;
    int i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("\nProcess %d\n", i+1);
        printf("Enter Process ID: ");
        scanf("%d", &pid[i]);

        printf("Enter Arrival Time: ");
        scanf("%d", &at[i]);

        printf("Enter Burst Time: ");
        scanf("%d", &bt[i]);
    }
```

```

    printf("Enter Priority: ");
    scanf("%d", &pr[i]);
}

while(completed_count < n)
{
    int highest_priority = 9999;
    int selected_process = -1;

    for(i = 0; i < n; i++)
    {
        if(at[i] <= current_time && completed[i] == 0)
        {
            if(pr[i] < highest_priority)
            {
                highest_priority = pr[i];
                selected_process = i;
            }
        }
    }

    if(selected_process == -1)
    {
        current_time++;
    }
    else
    {
        ct[selected_process] = current_time + bt[selected_process];
    }
}

```

```

    tat[selected_process] = ct[selected_process] - at[selected_process];
    wt[selected_process] = tat[selected_process] - bt[selected_process];

    current_time = ct[selected_process];
    completed[selected_process] = 1;
    completed_count++;
}
}

printf("\nProcess\tAT\tBT\tPriority\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t\t%d\t%d\t%d\n",
        pid[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
}

return 0;
}

```

```
"C:\Users\Admin\Desktop\CS" X + v
Enter number of processes: 5

Process 1
Enter Process ID: 1
Enter Arrival Time: 0
Enter Burst Time: 3
Enter Priority: 5

Process 2
Enter Process ID: 2
Enter Arrival Time: 2
Enter Burst Time: 2
Enter Priority: 3

Process 3
Enter Process ID: 3
Enter Arrival Time: 3
Enter Burst Time: 5
Enter Priority: 2

Process 4
Enter Process ID: 4
Enter Arrival Time: 4
Enter Burst Time: 4
Enter Priority: 4

Process 5
Enter Process ID: 5
Enter Arrival Time: 6
Enter Burst Time: 1
Enter Priority: 1

Process AT      BT      Priority      CT      TAT      WT
P1      0      3      5      3      3      0
P2      2      2      3      11     9      7
P3      3      5      2      8      5      0
P4      4      4      4      15     11     7
P5      6      1      1      9      3      2

Process returned 0 (0x0)   execution time : 29.375 s
Press any key to continue.
|
```

Priority scheduling Pre-emptive

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, choice;
```

```
    int pid[20], pr[20], at[20], bt[20], rt[20];
```

```
    int ct[20], tat[20], wt[20];
```

```
    int completed = 0, current_time = 0;
```

```
    int i, selected, best_priority;
```

```
    float avg_wt = 0, avg_tat = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter priorities (1 for higher number means higher priority, 2 for lower number means higher priority): ");
```

```
    scanf("%d", &choice);
```

```
    printf("Enter priorities:\n");
```

```
    for(i=0;i<n;i++)
```

```
        scanf("%d",&pr[i]);
```

```
    printf("Enter arrival times:\n");
```

```
    for(i=0;i<n;i++)
```

```

scanf("%d",&at[i]);

printf("Enter burst times:\n");
for(i=0;i<n;i++)
{
    scanf("%d",&bt[i]);
    rt[i] = bt[i];
    pid[i] = i+1;
}

while(completed < n)
{
    selected = -1;

    if(choice == 2)
        best_priority = 9999;
    else
        best_priority = -1;

    for(i=0;i<n;i++)
    {
        if(at[i] <= current_time && rt[i] > 0)
        {
            if(choice == 2)
            {
                if(pr[i] < best_priority)
                {
                    best_priority = pr[i];
                    selected = i;

```

```

    }
}
else
{
    if(pr[i] > best_priority)
    {
        best_priority = pr[i];
        selected = i;
    }
}
}

if(selected == -1)
{
    current_time++;
}
else
{
    rt[selected]--;
    current_time++;

    if(rt[selected] == 0)
    {
        ct[selected] = current_time;
        tat[selected] = ct[selected] - at[selected];
        wt[selected] = tat[selected] - bt[selected];

        avg_tat += tat[selected];
    }
}

```

```

        avg_wt += wt[selected];

        completed++;
    }
}

printf("\nPriority Scheduling (Pre-Emptive):\n");
printf("PID\tPrior\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i], pr[i], at[i], bt[i], ct[i], tat[i], wt[i], wt[i]);
}

printf("\nAverage turnaround time: %.6f\n", avg_tat/n);
printf("Average waiting time: %.6f\n", avg_wt/n);

return 0;
}

```



```

Enter number of processes: 7
Enter priorities (1 for higher number means higher priority, 2 for lower number means higher priority): 2
Enter priorities:
3
4
4
5
2
6
1
Enter arrival times:
0
1
3
4
5
6
10
Enter burst times:
8
2
4
1
6
5
1
Priority Scheduling (Pre-Emptive):
PID    Prior  AT    BT    CT    TAT    WT    RT
P1      3      0      8    15    15      7      7
P2      4      1      2    17    16     14     14
P3      4      3      4    21    18     14     14
P4      5      4      1    22    18     17     17
P5      2      5      6    12      7      1      1
P6      6      6      5    27    21     16     16
P7      1     10      1    11      1      0      0

Average turnaround time: 13.714286
Average waiting time: 9.857142

Process returned 0 (0x0)   execution time : 75.354 s
Press any key to continue.
|

```

Round Robin

```
#include <stdio.h>
```

```
int main()
```

```

{
    int n, tq;
    int at[20], bt[20], rt[20];
    int ct[20], tat[20], wt[20];
    int i, time = 0, remain, flag = 0;

    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Time Quantum: ");
    scanf("%d",&tq);

    printf("Enter arrival times:\n");
    for(i=0;i<n;i++)
        scanf("%d",&at[i]);

    printf("Enter burst times:\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&bt[i]);
        rt[i] = bt[i];
    }

    remain = n;

    for(time=0,i=0; remain!=0;)
    {

```

```

if(rt[i] <= tq && rt[i] > 0)
{
    time += rt[i];
    rt[i] = 0;
    flag = 1;
}
else if(rt[i] > 0)
{
    rt[i] -= tq;
    time += tq;
}

if(rt[i]==0 && flag==1)
{
    remain--;
    ct[i] = time;
    tat[i] = time - at[i];
    wt[i] = tat[i] - bt[i];

    avg_wt += wt[i];
    avg_tat += tat[i];

    flag = 0;
}

if(i == n-1)
    i = 0;
else if(at[i+1] <= time)
    i++;

```

```

        else
            i = 0;
    }

    printf("\nRRS scheduling:\n");
    printf("PID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

    for(i=0;i<n;i++)
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
    }

    printf("\nAverage turnaround time: %.6fms\n", avg_tat/n);
    printf("\nAverage waiting time: %.6fms\n", avg_wt/n);

    return 0;
}

```

```

Enter number of processes: 5
Enter Time Quantum: 2
Enter arrival times:
0
1
2
3
4
Enter burst times:
5
3
1
2
3

RRS scheduling:

```

PID	AT	BT	CT	TAT	WT	RT
1	0	5	14	14	9	0
2	1	3	12	11	8	0
3	2	1	5	3	2	0
4	3	2	7	4	2	0
5	4	3	13	9	6	0

```

Average turnaround time: 8.200000ms

Average waiting time: 5.400000ms

Process returned 0 (0x0)   execution time : 14.973 s
Press any key to continue.

```